

Question 1

```
3  #include <iostream>
4
5  class ArrayMultiplier
6  {
7  public:
8      // Pure virtual function
9      virtual void calculate() = 0;
10 };
11
12 class ArrayMultiplier1D : public ArrayMultiplier
13 {
14 private:
15     int* array;
16     int size;
17     int multiplier;
18
19 public:
20     // Constructor
21     ArrayMultiplier1D(int s, int mult)
22     {
23         size = s;
24         multiplier = mult;
25         array = new int[size];
26     }
27
28     // Destructor
29     ~ArrayMultiplier1D()
30     {
31         delete[] array;
32     }
33
34     // Function to input array values
35     void inputArray()
36     {
37         std::cout << "\nEnter " << size << " elements for 1D array:\n";
38         for (int i = 0; i < size; i++)
39         {
40             std::cout << "Element [" << i << "]: ";
41             std::cin >> array[i];
42         }
43     }
44
45     // Implementation of pure virtual function
46     void calculate()
47     {
48         std::cout << "\n--- 1D Array Multiplication ---";
49         std::cout << "\nOriginal Array: ";
50         for (int i = 0; i < size; i++)
51         {
52             std::cout << array[i] << " ";
53         }
54
55         std::cout << "\nMultiplier: " << multiplier;
56         std::cout << "\nResult: ";
57         for (int i = 0; i < size; i++)
58         {
59             std::cout << (array[i] * multiplier) << " ";
60         }
61         std::cout << "\n";
62     }
63 };
64
65 class ArrayMultiplier2D : public ArrayMultiplier
66 {
67 private:
68     int** array;
69     int rows;
70     int cols;
71     int multiplier;
72
73 public:
74     // Constructor
75     ArrayMultiplier2D(int r, int c, int mult)
```

```

74 // Constructor
75 ArrayMultiplier2D(int r, int c, int mult)
76 {
77     rows = r;
78     cols = c;
79     multiplier = mult;
80
81     array = new int*[rows];
82     for (int i = 0; i < rows; i++)
83     {
84         array[i] = new int[cols];
85     }
86 }
87
88 // Destructor
89 ~ArrayMultiplier2D()
90 {
91     for (int i = 0; i < rows; i++)
92     {
93         delete[] array[i];
94     }
95     delete[] array;
96 }
97
98 // Function to input array values
99 void inputArray()
100 {
101     std::cout << "\nEnter elements for " << rows << "x" << cols << " 2D array:\n";
102     for (int i = 0; i < rows; i++)
103     {
104         for (int j = 0; j < cols; j++)
105         {
106             std::cout << "Element [" << i << "][" << j << "]: ";
107             std::cin >> array[i][j];
108         }
109     }
110 }
111
112 // Implementation of pure virtual function
113 void calculate()
114 {
115     std::cout << "\n--- 2D Array Multiplication ---";
116     std::cout << "\nOriginal Array:\n";
117     for (int i = 0; i < rows; i++)
118     {
119         for (int j = 0; j < cols; j++)
120         {
121             std::cout << array[i][j] << " ";
122         }
123         std::cout << "\n";
124     }
125
126     std::cout << "\nMultiplier: " << multiplier;
127     std::cout << "\nResult:\n";
128     for (int i = 0; i < rows; i++)
129     {
130         for (int j = 0; j < cols; j++)
131         {
132             std::cout << (array[i][j] * multiplier) << " ";
133         }
134         std::cout << "\n";
135     }
136 }
137 };
138
139 int main()
140 {
141     std::cout << "=== Array Multiplier System ===\n";
142
143     // Testing 1D Array Multiplication
144     ArrayMultiplier1D arr1D(5, 3);
145     arr1D.inputArray();
146     arr1D.calculate();
147
148     // Testing 2D Array Multiplication
149     ArrayMultiplier2D arr2D(3, 3, 2);
150     arr2D.inputArray();
151     arr2D.calculate();
152
153     return 0;
154 }
155

```

=== Array Multiplier System ===

Enter 5 elements for 1D array:

Element [0]: 1

Element [1]: 2

Element [2]: 3

Element [3]: 4

Element [4]: 5

--- 1D Array Multiplication ---

Original Array: 1 2 3 4 5

Multiplier: 3

Result: 3 6 9 12 15

Enter elements for 3x3 2D array:

Element [0][0]: 1

Element [0][1]: 2

Element [0][2]: 4

Element [1][0]: 3

Element [1][1]: 5

Element [1][2]: 6

Element [2][0]: 7

Element [2][1]: 8

Element [2][2]: 9

--- 2D Array Multiplication ---

Original Array:

1 2 4

3 5 6

7 8 9

Multiplier: 2

Result:

2 4 8

6 10 12

14 16 18

Question 2.

```
3 #include <iostream>
4
5 class Store
6 {
7     protected:
8         float total_bill;
9
10    public:
11        // Constructor
12        Store(float bill)
13        {
14            total_bill = bill;
15        }
16
17        // Pure virtual function
18        virtual float calculateFinalBill() = 0;
19
20        // Function to display bill details
21        virtual void displayBill() = 0;
22 };
23
24 class ImtiazStore : public Store
25 {
26     private:
27         float discount;
28
29    public:
30        // Constructor
31        ImtiazStore(float bill) : Store(bill)
32        {
33            discount = 0.07; // 7% discount
34        }
35
36        // Implementation of pure virtual function
37        float calculateFinalBill()
38        {
39            float discountAmount = total_bill * discount;
40            float finalBill = total_bill - discountAmount;
41            return finalBill;
42        }
43
44        // Display bill details
45        void displayBill()
46        {
47            std::cout << "\n--- Imtiaz Store Bill ---";
48            std::cout << "\nTotal Bill: " << total_bill << " PKR";
49            std::cout << "\nDiscount: " << (discount * 100) << "%";
50            std::cout << "\nDiscount Amount: " << (total_bill * discount) << " PKR";
51            std::cout << "\nFinal Bill: " << calculateFinalBill() << " PKR";
52            std::cout << "\n";
53        }
54 };
55
56 class BinHashimStore : public Store
57 {
58     private:
59         float discount;
60
61    public:
62        // Constructor
63        BinHashimStore(float bill) : Store(bill)
64        {
65            discount = 0.05; // 5% discount
66        }
67
68        // Implementation of pure virtual function
69        float calculateFinalBill()
70        {
71            float discountAmount = total_bill * discount;
72            float finalBill = total_bill - discountAmount;
73            return finalBill;
74        }
75
76        // Display bill details
77        void displayBill()
78        {
79            std::cout << "\n--- Bin Hashim Store Bill ---";
80            std::cout << "\nTotal Bill: " << total_bill << " PKR";
81            std::cout << "\nDiscount: " << (discount * 100) << "%";
82            std::cout << "\nDiscount Amount: " << (total_bill * discount) << " PKR";
83            std::cout << "\nFinal Bill: " << calculateFinalBill() << " PKR";
84            std::cout << "\n";
85        }
86 };
87
```

```

85 };
86 };
87
88 int main()
89 {
90     float billAmount;
91
92     std::cout << "=== Store Discount System ===\n";
93     std::cout << "\nEnter total bill amount: ";
94     std::cin >> billAmount;
95
96     // Creating store objects
97     ImtiazStore imtiaz(billAmount);
98     BinHashimStore binHashim(billAmount);
99
100    // Displaying bills for both stores
101    imtiaz.displayBill();
102    binHashim.displayBill();
103
104    // Using polymorphism with base class pointer
105    std::cout << "\n\n--- Using Polymorphism ---";
106    Store* storePtr;
107
108    storePtr = &imtiaz;
109    std::cout << "\nImtiaz Store Final Bill: " << storePtr->calculateFinalBill() << " PKR";
110
111    storePtr = &binHashim;
112    std::cout << "\nBin Hashim Store Final Bill: " << storePtr->calculateFinalBill() << " PKR";
113    std::cout << "\n";
114
115    return 0;
116 }
117

```

=== Store Discount System ===

Enter total bill amount: 1500

--- Imtiaz Store Bill ---

Total Bill: 1500 PKR

Discount: 7%

Discount Amount: 105 PKR

Final Bill: 1395 PKR

--- Bin Hashim Store Bill ---

Total Bill: 1500 PKR

Discount: 5%

Discount Amount: 75 PKR

Final Bill: 1425 PKR

--- Using Polymorphism ---

Imtiaz Store Final Bill: 1395 PKR

Bin Hashim Store Final Bill: 1425 PKR

Question 3.

```
2
3 #include <iostream>
4 #include <string.h>
5
6 // Forward declarations
7 class Vehicle;
8 class RentalSystem;
9
10 // Abstract Base Class
11 class Vehicle
12 {
13     protected:
14         int carId;
15         char* brand;
16         char* model;
17         bool available;
18
19     public:
20         // Constructor
21         Vehicle()
22         {
23             carId = 0;
24             brand = NULL;
25             model = NULL;
26             available = true;
27         }
28
29         // Destructor
30         virtual ~Vehicle()
31         {
32             if (brand != NULL)
33             {
34                 delete[] brand;
35             }
36             if (model != NULL)
37             {
38                 delete[] model;
39             }
40         }
41
42         // Pure virtual functions
43         virtual bool isAvailable() = 0;
44         virtual void rent() = 0;
45         virtual void returnVehicle() = 0;
46         virtual void displayInfo() = 0;
47     };
48
49 // Derived Class: Car
50 class Car : public Vehicle
51 {
52     public:
53         // Constructor
54         Car(int id, const char* b, const char* m)
55         {
56             carId = id;
57
58             brand = new char[strlen(b) + 1];
59             strcpy(brand, b);
60
61             model = new char[strlen(m) + 1];
62             strcpy(model, m);
63
64             available = true;
65         }
66
67         // Implementation of isAvailable
68         bool isAvailable()
69         {
70             return available;
71         }
72
73         // Implementation of rent
74         void rent()
75         {
76             if (available)
77             {
78                 available = false;
79                 std::cout << "\nCar ID " << carId << " (" << brand << " " << model << ") has been rented.";
80             }
81             else
82             {
83                 std::cout << "\nCar ID " << carId << " is not available!";
84             }
85         }
86
87         // Implementation of returnVehicle
88         void returnVehicle()
89         {
90         }
```

```

85     }
86
87     // Implementation of returnVehicle
88     void returnVehicle()
89     {
90         if (!available)
91         {
92             available = true;
93             std::cout << "\nCar ID " << carId << " (" << brand << " " << model << ") has been returned.";
94         }
95         else
96         {
97             std::cout << "\nCar ID " << carId << " was not rented!";
98         }
99     }
100
101     // Display car information
102     void displayInfo()
103     {
104         std::cout << "\nCar ID: " << carId;
105         std::cout << "\nBrand: " << brand;
106         std::cout << "\nModel: " << model;
107         std::cout << "\nStatus: " << (available ? "Available" : "Rented");
108     }
109
110     int getId()
111     {
112         return carId;
113     }
114 };
115
116 // Rental System Class
117 class RentalSystem
118 {
119 public:
120     void rentVehicle(Vehicle* v)
121     {
122         if (v->isAvailable())
123         {
124             v->rent();
125         }
126         else
127         {
128             std::cout << "\nVehicle is not available for rent!";
129         }
130     }
131
132     void returnVehicle(Vehicle* v)
133     {
134         v->returnVehicle();
135     }
136
137     void checkAvailability(Vehicle* v)
138     {
139         if (v->isAvailable())
140         {
141             std::cout << "\nVehicle is available for rent.";
142         }
143         else
144         {
145             std::cout << "\nVehicle is currently rented.";
146         }
147     }
148 };
149
150 // Customer Class
151 class Customer
152 {
153 private:
154     char* name;
155     int customerId;
156
157 public:
158     // Constructor
159     Customer(int id, const char* n)
160     {
161         customerId = id;
162         name = new char[strlen(n) + 1];
163         strcpy(name, n);
164     }
165
166     // Destructor
167     ~Customer()
168     {
169         delete[] name;
170     }
171
172     void rentVehicle(Vehicle* v, RentalSystem* rs)

```



```

167 ~Customer()
168 {
169     delete[] name;
170 }
171
172 void rentVehicle(Vehicle* v, RentalSystem* rs)
173 {
174     std::cout << "\n\nCustomer " << name << " (ID: " << customerId << ") wants to rent a vehicle.";
175     rs->rentVehicle(v);
176 }
177
178 void returnVehicle(Vehicle* v, RentalSystem* rs)
179 {
180     std::cout << "\n\nCustomer " << name << " (ID: " << customerId << ") is returning a vehicle.";
181     rs->returnVehicle(v);
182 }
183 };
184
185 int main()
186 {
187     std::cout << "=== Car Rental System ===\n";
188
189     // Creating array of pointers to Vehicle (base class)
190     Vehicle* vehicles[5];
191
192     // Dynamic memory allocation for Car objects
193     vehicles[0] = new Car(101, "Toyota", "Corolla");
194     vehicles[1] = new Car(102, "Honda", "Civic");
195     vehicles[2] = new Car(103, "Suzuki", "Alto");
196     vehicles[3] = new Car(104, "Hyundai", "Elantra");
197     vehicles[4] = new Car(105, "Kia", "Sportage");
198
199     // Displaying all vehicles
200     std::cout << "\n--- Available Vehicles ---";
201     for (int i = 0; i < 5; i++)
202     {
203         std::cout << "\n";
204         vehicles[i]->displayInfo();
205     }
206
207     // Creating rental system
208     RentalSystem rentalSystem;
209
210     // Creating customers
211     Customer customer1(1, "Ali Ahmed");
212     Customer customer2(2, "Sara Khan");
213
214     // Customer 1 rents a vehicle
215     customer1.rentVehicle(vehicles[0], &rentalSystem);
216
217     // Customer 2 rents a vehicle
218     customer2.rentVehicle(vehicles[1], &rentalSystem);
219
220     // Display updated status
221     std::cout << "\n\n--- Updated Vehicle Status ---";
222     for (int i = 0; i < 5; i++)
223     {
224         std::cout << "\n";
225         vehicles[i]->displayInfo();
226     }
227
228     // Customer 1 returns vehicle
229     customer1.returnVehicle(vehicles[0], &rentalSystem);
230
231     // Customer 2 tries to rent already rented vehicle
232     Customer customer3(3, "Hassan Raza");
233     customer3.rentVehicle(vehicles[1], &rentalSystem);
234
235     // Display final status
236     std::cout << "\n\n--- Final Vehicle Status ---";
237     for (int i = 0; i < 5; i++)
238     {
239         std::cout << "\n";
240         vehicles[i]->displayInfo();
241     }
242
243     // Cleaning up dynamically allocated memory
244     for (int i = 0; i < 5; i++)
245     {
246         delete vehicles[i];
247     }
248
249     std::cout << "\n";
250     return 0;
251 }
252

```

=== Car Rental System ===

--- Available Vehicles ---

Car ID: 101
Brand: Toyota
Model: Corolla
Status: Available

Car ID: 102
Brand: Honda
Model: Civic
Status: Available

Car ID: 103
Brand: Suzuki
Model: Alto
Status: Available

Car ID: 104
Brand: Hyundai
Model: Elantra
Status: Available

Car ID: 105
Brand: Kia
Model: Sportage
Status: Available

Customer Ali Ahmed (ID: 1) wants to rent a vehicle.
Car ID 101 (Toyota Corolla) has been rented.

Customer Sara Khan (ID: 2) wants to rent a vehicle.
Car ID 102 (Honda Civic) has been rented.

--- Updated Vehicle Status ---

Car ID: 101
Brand: Toyota
Model: Corolla
Status: Rented

Car ID: 102
Brand: Honda
Model: Civic
Status: Rented

Car ID: 103
Brand: Suzuki
Model: Alto
Status: Available

Car ID: 103
Brand: Suzuki
Model: Alto
Status: Available

Car ID: 104
Brand: Hyundai
Model: Elantra
Status: Available

Car ID: 105
Brand: Kia
Model: Sportage
Status: Available

Customer Ali Ahmed (ID: 1) is returning a vehicle.
Car ID 101 (Toyota Corolla) has been returned.

Customer Hassan Raza (ID: 3) wants to rent a vehicle.
Vehicle is not available for rent!

--- Final Vehicle Status ---

Car ID: 101
Brand: Toyota
Model: Corolla
Status: Available

Car ID: 102
Brand: Honda
Model: Civic
Status: Rented

Car ID: 103
Brand: Suzuki
Model: Alto
Status: Available

Car ID: 104
Brand: Hyundai
Model: Elantra
Status: Available

Car ID: 105
Brand: Kia
Model: Sportage

Question 4.

```
3  #include <iostream>
4  #include <string.h>
5
6  class EncryptionTechnique
7  {
8  protected:
9      char* message;
10     char* encryptedMessage;
11
12 public:
13     // Constructor
14     EncryptionTechnique()
15     {
16         message = NULL;
17         encryptedMessage = NULL;
18     }
19
20     // Destructor
21     virtual ~EncryptionTechnique()
22     {
23         if (message != NULL)
24         {
25             delete[] message;
26         }
27         if (encryptedMessage != NULL)
28         {
29             delete[] encryptedMessage;
30         }
31     }
32
33     // Function to set message
34     void setMessage(const char* msg)
35     {
36         if (message != NULL)
37         {
38             delete[] message;
39         }
40         message = new char[strlen(msg) + 1];
41         strcpy(message, msg);
42     }
43
44     // Pure virtual function
45     virtual void encrypt() = 0;
46
47     // Display encrypted message
48     virtual void displayEncrypted() = 0;
49 };
50
51 class EncryptionTechnique1 : public EncryptionTechnique
52 {
53 public:
54     // Implementation of encrypt function
55     void encrypt()
56     {
57         if (message == NULL)
58         {
59             return;
60         }
61
62         // Calculate size needed for encrypted message
63         int len = strlen(message);
64         int encryptedSize = len * 3 + 1; // max 3 digits per character + null terminator
65
66         if (encryptedMessage != NULL)
67         {
68             delete[] encryptedMessage;
69         }
70         encryptedMessage = new char[encryptedSize];
71         encryptedMessage[0] = '\0';
72
73         // Convert each character to ASCII code
74         char temp[10];
75         for (int i = 0; i < len; i++)
76         {
77             sprintf(temp, "%d", (int)message[i]);
78             strcat(encryptedMessage, temp);
79         }
80     }
81
82     // Display encrypted message
83     void displayEncrypted()
84     {
85         std::cout << "\n--- Encryption Technique 1 ---";
86         std::cout << "\nOriginal Message: " << message;
87         std::cout << "\nEncryption Method: Convert alphabets to ASCII codes";
88         std::cout << "\nEncrypted Message: " << encryptedMessage;
89         std::cout << "\n";
90     }
91 }
```

```

89         std::cout << "\n";
90     }
91 };
92
93 class EncryptionTechnique2 : public EncryptionTechnique
94 {
95     public:
96         // Implementation of encrypt function
97         void encrypt()
98         {
99             if (message == NULL)
100             {
101                 return;
102             }
103
104             // Calculate size needed for encrypted message
105             int len = strlen(message);
106             int encryptedSize = len * 3 + 1; // max 3 digits per character + null terminator
107
108             if (encryptedMessage != NULL)
109             {
110                 delete[] encryptedMessage;
111             }
112             encryptedMessage = new char[encryptedSize];
113             encryptedMessage[0] = '\0';
114
115             // Convert each character to ASCII code + 2
116             char temp[10];
117             for (int i = 0; i < len; i++)
118             {
119                 sprintf(temp, "%d", (int)message[i] + 2);
120                 strcat(encryptedMessage, temp);
121             }
122         }
123
124         // Display encrypted message
125         void displayEncrypted()
126         {
127             std::cout << "\n--- Encryption Technique 2 ---";
128             std::cout << "\nOriginal Message: " << message;
129             std::cout << "\nEncryption Method: Convert alphabets to ASCII codes + 2";
130             std::cout << "\nEncrypted Message: " << encryptedMessage;
131             std::cout << "\n";
132         }
133 };
134
135 int main()
136 {
137     char inputMessage[100];
138
139     std::cout << "=== Encryption System ===\n";
140     std::cout << "\nEnter a message to encrypt: ";
141     std::cin.getline(inputMessage, 100);
142
143     // Creating encryption objects
144     EncryptionTechnique1 technique1;
145     EncryptionTechnique2 technique2;
146
147     // Setting message for both techniques
148     technique1.setMessage(inputMessage);
149     technique2.setMessage(inputMessage);
150
151     // Encrypting using both techniques
152     technique1.encrypt();
153     technique2.encrypt();
154
155     // Displaying encrypted messages
156     technique1.displayEncrypted();
157     technique2.displayEncrypted();
158
159     // Using polymorphism with base class pointer
160     std::cout << "\n--- Using Polymorphism ---";
161     EncryptionTechnique* encPtr;
162
163     encPtr = &technique1;
164     std::cout << "\nUsing Technique 1:";
165     encPtr->displayEncrypted();
166
167     encPtr = &technique2;
168     std::cout << "\nUsing Technique 2:";
169     encPtr->displayEncrypted();
170
171     return 0;
172 }
173

```

=== Encryption System ===

Enter a message to encrypt: Hello World

--- Encryption Technique 1 ---

Original Message: Hello World

Encryption Method: Convert alphabets to ASCII codes

Encrypted Message: 721011081081113287111114108100

--- Encryption Technique 2 ---

Original Message: Hello World

Encryption Method: Convert alphabets to ASCII codes + 2

Encrypted Message: 741031101101133489113116110102

--- Using Polymorphism ---

Using Technique 1:

--- Encryption Technique 1 ---

Original Message: Hello World

Encryption Method: Convert alphabets to ASCII codes

Encrypted Message: 721011081081113287111114108100

Using Technique 2:

--- Encryption Technique 2 ---

Original Message: Hello World

Encryption Method: Convert alphabets to ASCII codes + 2

Encrypted Message: 741031101101133489113116110102

Question 5.

```
1 // Exercise 5: Decryption techniques using abstract class and pure virtual function
```

```
2  
3 #include <iostream>  
4 #include <string.h>  
5 #include <stdlib.h>
```

```
6  
7 class DecryptionTechnique
```

```
8 {
```

```
9     protected:
```

```
10         char* encryptedMessage;  
11         char* decryptedMessage;
```

```
12  
13     public:
```

```
14         // Constructor
```

```
15         DecryptionTechnique()
```

```
16     {
```

```
17         encryptedMessage = NULL;  
18         decryptedMessage = NULL;
```

```
19     }
```

```
20  
21         // Destructor
```

```
22         virtual ~DecryptionTechnique()
```

```
23     {
```

```
24         if (encryptedMessage != NULL)  
25         {  
26             delete[] encryptedMessage;  
27         }  
28         if (decryptedMessage != NULL)  
29         {  
30             delete[] decryptedMessage;  
31         }  
32     }
```

```
33  
34         // Function to set encrypted message
```

```
35         void setEncryptedMessage(const char* msg)
```

```
36     {
```

```
37         if (encryptedMessage != NULL)  
38         {  
39             delete[] encryptedMessage;  
40         }  
41         encryptedMessage = new char[strlen(msg) + 1];  
42         strcpy(encryptedMessage, msg);  
43     }
```

```
44  
45         // Pure virtual function
```

```
46         virtual void decrypt() = 0;
```

```
47  
48         // Display decrypted message
```

```
49         virtual void displayDecrypted() = 0;
```

```
50     };
```

```
51  
52     class DecryptionTechnique1 : public DecryptionTechnique
```

```
53     {
```

```
54     public:
```

```
55         // Implementation of decrypt function
```

```
56         void decrypt()
```

```
57     {
```

```
58         if (encryptedMessage == NULL)  
59         {  
60             return;  
61         }
```

```
62  
63         int len = strlen(encryptedMessage);
```

```
64         if (decryptedMessage != NULL)
```

```
65         {  
66             delete[] decryptedMessage;
```

```
67         }  
68         decryptedMessage = new char[len + 1];
```

```
69  
70         int decIndex = 0;
```

```
71         int i = 0;
```

```
72  
73         // Parse encrypted message and convert back to characters
```

```
74         while (i < len)
```

```
75     {
```

```
76         char numStr[4] = {0};  
77         int numIndex = 0;
```

```
78  
79         // Read 2-3 digits for ASCII code
```

```
80         while (i < len && numIndex < 3)
```

```
81     {
```

```
82         numStr[numIndex] = encryptedMessage[i];  
83         numIndex++;  
84         i++;
```

```
85  
86         // Check if we have a valid ASCII code
```

```
87         int asciiCode = atoi(numStr);  
88         if (asciiCode >= 32 && asciiCode <= 126)
```

```

87         int asciiCode = atoi(numStr);
88         if (asciiCode >= 32 && asciiCode <= 126)
89         {
90             // Valid printable ASCII character
91             decryptedMessage[decIndex] = (char)asciiCode;
92             decIndex++;
93             break;
94         }
95     }
96 }
97
98     decryptedMessage[decIndex] = '\0';
99 }
100
101 // Display decrypted message
102 void displayDecrypted()
103 {
104     std::cout << "\n--- Decryption Technique 1 ---";
105     std::cout << "\nEncrypted Message: " << encryptedMessage;
106     std::cout << "\nDecryption Method: Convert ASCII codes back to alphabets";
107     std::cout << "\nDecrypted Message: " << decryptedMessage;
108     std::cout << "\n";
109 }
110 };
111
112 class DecryptionTechnique2 : public DecryptionTechnique
113 {
114 public:
115     // Implementation of decrypt function
116     void decrypt()
117     {
118         if (encryptedMessage == NULL)
119         {
120             return;
121         }
122
123         int len = strlen(encryptedMessage);
124         if (decryptedMessage != NULL)
125         {
126             delete[] decryptedMessage;
127         }
128         decryptedMessage = new char[len + 1];
129
130         int decIndex = 0;
131         int i = 0;
132
133         // Parse encrypted message and convert back to characters
134         while (i < len)
135         {
136             char numStr[4] = {0};
137             int numIndex = 0;
138
139             // Read 2-3 digits for ASCII code
140             while (i < len && numIndex < 3)
141             {
142                 numStr[numIndex] = encryptedMessage[i];
143                 numIndex++;
144                 i++;
145
146                 // Check if we have a valid ASCII code (subtract 2 first)
147                 int asciiCode = atoi(numStr) - 2;
148                 if (asciiCode >= 32 && asciiCode <= 126)
149                 {
150                     // Valid printable ASCII character
151                     decryptedMessage[decIndex] = (char)asciiCode;
152                     decIndex++;
153                     break;
154                 }
155             }
156
157             decryptedMessage[decIndex] = '\0';
158         }
159
160         // Display decrypted message
161         void displayDecrypted()
162         {
163             std::cout << "\n--- Decryption Technique 2 ---";
164             std::cout << "\nEncrypted Message: " << encryptedMessage;
165             std::cout << "\nDecryption Method: Convert ASCII codes - 2 back to alphabets";
166             std::cout << "\nDecrypted Message: " << decryptedMessage;
167             std::cout << "\n";
168         }
169     }
170 };
171
172 int main()
173 {
174     char encryptedMsg[200];

```



```
171
172 int main()
173 {
174     char encryptedMsg1[200];
175     char encryptedMsg2[200];
176
177     std::cout << "=== Decryption System ===\n";
178
179     std::cout << "\nEnter encrypted message (Technique 1): ";
180     std::cin.getline(encryptedMsg1, 200);
181
182     std::cout << "Enter encrypted message (Technique 2): ";
183     std::cin.getline(encryptedMsg2, 200);
184
185     // Creating decryption objects
186     DecryptionTechnique1 technique1;
187     DecryptionTechnique2 technique2;
188
189     // Setting encrypted messages
190     technique1.setEncryptedMessage(encryptedMsg1);
191     technique2.setEncryptedMessage(encryptedMsg2);
192
193     // Decrypting using both techniques
194     technique1.decrypt();
195     technique2.decrypt();
196
197     // Displaying decrypted messages
198     technique1.displayDecrypted();
199     technique2.displayDecrypted();
200
201     // Using polymorphism with base class pointer
202     std::cout << "\n--- Using Polymorphism ---";
203     DecryptionTechnique* decPtr;
204
205     decPtr = &technique1;
206     std::cout << "\nUsing Technique 1:";
207     decPtr->displayDecrypted();
208
209     decPtr = &technique2;
210     std::cout << "\nUsing Technique 2:";
211     decPtr->displayDecrypted();
212
213     std::cout << "\n\n--- Example ---";
214     std::cout << "\nFor 'Hello'";
215     std::cout << "\nEncrypted (Technique 1): 7269767679";
216     std::cout << "\nEncrypted (Technique 2): 7471787881";
217     std::cout << "\n";
218
219     return 0;
220 }
221
```

=== Decryption System ===

Enter encrypted message (Technique 1): 72101108108111

Enter encrypted message (Technique 2): 74103110110113

--- Decryption Technique 1 ---

Encrypted Message: 72101108108111

Decryption Method: Convert ASCII codes back to alphabets

Decrypted Message: Hello

--- Decryption Technique 2 ---

Encrypted Message: 74103110110113

Decryption Method: Convert ASCII codes - 2 back to alphabets

Decrypted Message: Hello

--- Using Polymorphism ---

Using Technique 1:

--- Decryption Technique 1 ---

Encrypted Message: 72101108108111

Decryption Method: Convert ASCII codes back to alphabets

Decrypted Message: Hello

Using Technique 2:

--- Decryption Technique 2 ---

Encrypted Message: 74103110110113

Decryption Method: Convert ASCII codes - 2 back to alphabets

Decrypted Message: Hello

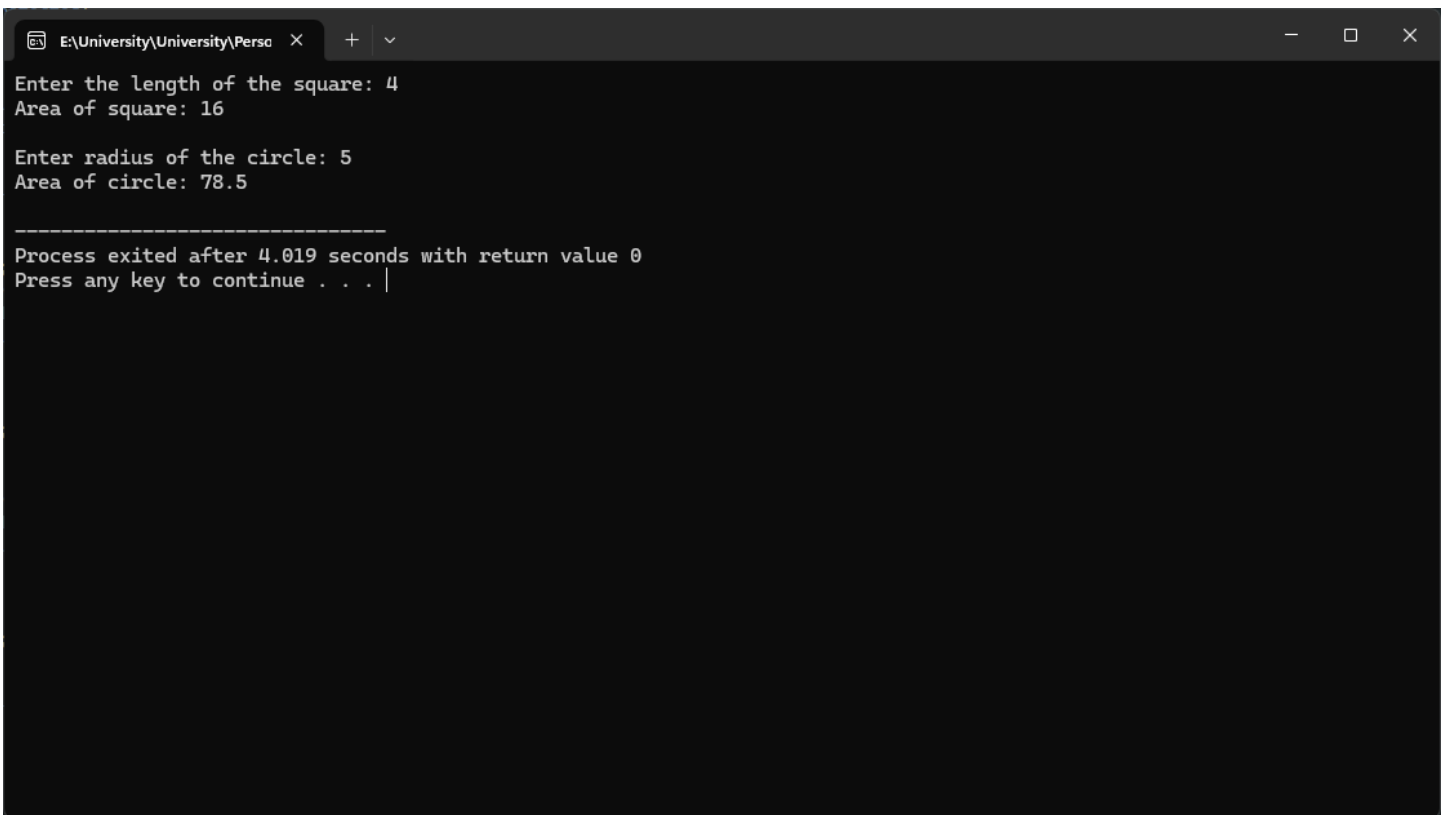
--- Example ---

For 'Hello'

Encrypted (Technique 1): 7269767679

Encrypted (Technique 2): 7471787881

Example



```
E:\University\University\Perso X + v
Enter the length of the square: 4
Area of square: 16

Enter radius of the circle: 5
Area of circle: 78.5

-----
Process exited after 4.019 seconds with return value 0
Press any key to continue . . . |
```